

1 Check in

Discuss questions, if you have any, with the tutor and the rest of the class, about the material and content so far.

2 Introduction

Today we're doing a mix of algorithms and data structure problems. A recurring theme will be the tradeoffs between processing *static* datasets versus *dynamic* ones. Static datasets are when you'll be given the entire set of data beforehand, and dynamic ones are when new pieces of data might come in at any moment, but we might want to perform intermediate operations in the meantime. We will start with a short problem on k largest elements to practice applying some algorithms and data structures seen in the lectures so far. Then move onto new trees (probably not seen before) to practice designing, and implementing new trees.

3 Problems

Problem 1. Warm Up – k largest on static datasets

Given a data set containing n unique elements, return the largest k elements.

Your solution must be **better** than $O(n \log n)$. Note that:

- You do not have to return the elements in sorted order
- Your solution does not have to be deterministic (*wink wink*)

Problem 2. k largest on dynamic datasets

Instead of being given all the elements at once, we now don't know all the elements in advance. Whenever we see a new element, we need to incorporate it into our dataset. We must also still be ready to answer queries for the largest k elements that we have seen *so far*. Note that the k here is fixed beforehand and will never change.

Design a data structure that supports the following two operations:

- `insertNext(x)`: add a new item x into the data structure.
- `getKLargest()`: return the current largest k elements in the data structure. You do not have to return them in sorted order.

You are guaranteed that all inserted elements are unique, and `getKLargest()` is only called when there are at least k elements.

See if you can come up with a solution that, if we have seen n elements so far:

- Uses $O(\log n)$ time for insertion, $O(n)$ space, and $O(k)$ time to list out the largest k elements.
- Uses $O(\log k)$ time for insertion, $O(k)$ space, and $O(k)$ time to list out the largest k elements.

Problem 3. k-d trees

Let's move to building a type of tree for storing points in k dimensions, called a k - d tree (short for k -dimensional tree). For simplicity, we're going to deal with just 2 dimensions (i.e. $k = 2$). Also, we are going to assume that our n points in 2D space are given to us upfront and will never change. In particular, this is a static dataset.

A k - d tree internal node stores the following data:

- A splitting line, and an indicator on whether it is horizontal or vertical.
- If the splitting line is horizontal, then it holds an “up” subtree reference/pointer and a “down” subtree reference/pointer.
- Otherwise, the splitting line is vertical, then it holds a “left” subtree reference/pointer and a “right” subtree reference/pointer.

A k - d tree leaf node stores simply:

- The coordinate of a single point (x, y) .

The idea behind a k - d tree is that it needs to support the following operations efficiently:

- `lookup( $x, y$ )`: Should return *true* if there is indeed a point stored at those coordinates, and *false* otherwise.
- `nearest-neighbour( $x, y$ )`: Should return the nearest stored point to the query coordinates (x, y) .
- `range-query( $R$ )`: Should return the number of points that are within the rectangle R .

To do this, the k - d tree (for the special case of $k = 2$) has the following rough invariants for the internal nodes:

- If the node contains a horizontal splitting line $y = c$, then:
 - All points (x, y) in the “up” subtree must be above the splitting line ($y \geq c$)
 - All points (x, y) in the “down” subtree must be below the splitting line ($y < c$)
- Similarly, if the node contains a vertical splitting line $x = c$, then:

- All points (x, y) in the “right” subtree must be to the right of the splitting line ($x \geq c$)
- All points (x, y) in the “left” subtree must be to the left of the splitting line ($x < c$)
- If a node contains a horizontal splitting line, the immediate subtrees should contain a vertical splitting line. Otherwise, the node contains a vertical splitting line, then the immediate subtrees should contain a horizontal splitting line.
- **Importantly**, at every internal node, the number of leaves in both subtrees should be about the same (or almost half each).

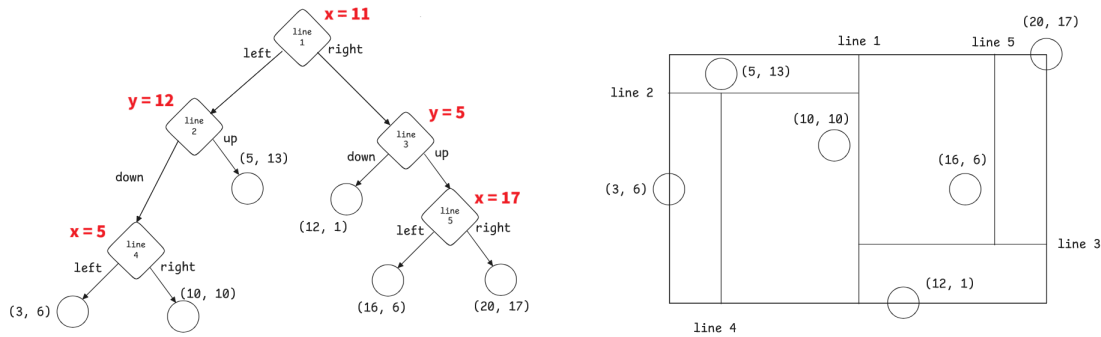


Figure 1: A bunch of points and how it might be stored

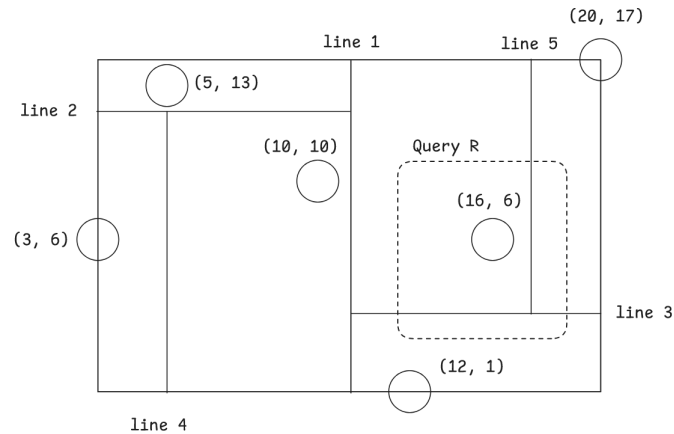
Side note: k - d trees are actually great for applications in AI and searching. When you map your data (e.g. websites) into some k -dimensional space, and if you can map a query like a Google search string into a point in that space. Then intuitively, $\text{nearest-neighbour}(x, y)$ should give you the most relevant search result. Grant Sanderson of 3Blue1Brown has a great video that touches on this: <https://www.youtube.com/watch?v=wjZofJX0v4M>.

In this problem, we’ll give you a quick taste of k - d trees, learn how to build them, and how to handle the range query.

Problem 3.a. Given an array of n points in 2D space, design an algorithm that builds a balanced k - d tree such that each internal node stores roughly half the points in its left subtree and half the points in its right subtree. Ideally, do so in expected $O(n \log n)$ time. (If you want a small challenge, it’s not hard to also do this in worst-case $O(n \log n)$ time. It’s not hard, just a little annoying to implement).

Problem 3.b. Given a k - d tree of n points and query coordinates (x, y) , design an algorithm that reports whether there is indeed a point at (x, y) . The algorithm should run in $O(\log n)$ time.

Problem 3.c. Given a k - d tree of n points and a query rectangle R , design an efficient algorithm that counts the number of points within the rectangle R .



For example, this query should return 1, as there is a single point (16, 6) contained in R .

This one might be a little tricky at first, so here's a little help in making a recursive approach. Assume that our internal nodes all come with the following:

- Number of points (leaf nodes) in the subtree
- Minimum and maximum x -coordinate across all points in the subtree
- Minimum and maximum y -coordinate across all points in the subtree

Notice then for any node, we know how many points are in its subtree, and a rectangular region that contains all points.

Then think about what you should do with the following cases:

1. If R *completely* contains the node's region, what should you do?
2. If R *partially* intersects the node's region, what should you do?
3. If R *does not* intersect the node's region, what should you do?

Problem 3.d.

Next, let's show that your algorithm runs in $O(\sqrt{n})$ time. Pretty neat, huh?

Your algorithm should only visit a node if its region intersects (either partially or completely) with the query rectangle R . In order for a region to intersect with R , the region must intersect with at

least one of R 's edges.

We'll consider each edge individually. To make our lives easier, let's extend the edges into complete lines. What we compute next will be an upper bound for the runtime.

- (i) Consider a vertical line L with equation $x = c$. Show that, for a subtree containing n points, the number of regions within this subtree that intersect L satisfies the recurrence:

$$T(n) = 2T\left(\frac{n}{4}\right) + 1$$

- (ii) Show that the recurrence solves to $O(\sqrt{n})$.

Problem 4. Midterm Mayhem!

Midterms are approaching, and everyone is panicking. Maxwell and Minnie are having a heated argument over a past-year problem. Help them settle the debate and restore their relationship!

(*Totally Real*) Past-Year Problem

Usually, we can say $x < y$ if x and y are both numbers. However, we can extend the idea of "less-than" for strings as well, by using lexicographical ordering.

For the rest of this problem, all strings only have lowercase English letters.

Given two strings s and t , we say s is less than t ($s < t$) if and only if one of the following is true:

- s is a *proper* prefix of t
(e.g. "java" < "javascript")
- At the first position i where s and t differ ($s_i \neq t_i$), the letter s_i comes before t_i alphabetically
(e.g. "javascript" < "jasasucks")

From here, we can also say s is less than or equal to t ($s \leq t$) if and only if $s < t$ or $s = t$.

Now, your task is to build a data structure that supports the following operations:

- **insert**(t): Insert a string t . The same string may be inserted multiple times.
- **rank**(t): If t has been inserted at least once, return the number of strings s in the data structure such that $s < t$ (including duplicates). Otherwise, return -1 .

For example, starting with an empty data structure, if we call **insert**("a"), **insert**("b"), **insert**("b"), **insert**("a"), then calling **rank**("b") now should return 2.

Problem 4.a.

Maxwell: "What are you talking about? This is obviously an AVL problem!"

Minnie: "But AVLs don't have duplicates!"

Maxwell: "Then just augment it!"

Show how you can augment an AVL tree to solve this problem. If there are N strings with length at most L , what is the worst-case time complexity of each operation?

Problem 4.b.

Minnie: "You dummy, there's no way it's that easy! This problem is worth 10 marks!"

Maxwell: "Do you have a better solution then?"

Minnie: "I don't... I don't know, use a trie or something?!"

Maxwell: "How?!"

Show how you can instead use a trie to solve this problem. Now, what is the worst-case time complexity of each operation?

Problem 4.c.

Maxwell: "Okay, okay, fine. I'm sorry for yelling at you."

Minnie: "Hmph!"

Maxwell: "Can we just move on to the next part?"

Show how you can implement this new operation:

- **countBetween(l, r):** If l and r have been inserted at least once, return the number of strings s in the data structure such that $l \leq s < r$ (including duplicates). Otherwise, return -1 .

Problem 5. (Optional) Give this problem a Trie

Let's introduce a new operation called the bitwise XOR. For the rest of this problem, you may assume all numbers are positive 32-bit integers.

As an example, for $a = 314159265$ and $b = 271828182$, their binary representations are:

a = 00010010101110011011000010100001

b = 00010000001100111100010011010110

Then, for each pair of corresponding bits, we take their logical XOR. In other words, if both bits are the same, the result is 0, and if both bits are different, the result is 1. In this case, we get:

x = 00000010100010100111010001110111

Converting back to decimal, we get $a \text{ XOR } b = 42628215$.

Now here's your task: Given an array of positive 32-bit integers, find 2 numbers such that their XOR is maximum.

(Hint: Construct the result one bit at a time, starting from the most significant bit)